

1. Environment Setup and Ingestion Utilities

This section initializes the AWS environment, installs required libraries, and defines helper functions for reading raw SDWA CSV files and writing them to S3 as Parquet datasets with a stable STRING schema. These utilities support both normal and streaming ingestion paths.

```
In [1]: # =====
# Week 3 – Environment Setup + Ingestion Utilities
# =====

# Install required packages
%pip install --force-reinstall --no-deps sagemaker==2.218.0
%pip install --no-deps ppft
%pip install --no-deps pathos

import time
import boto3
import pandas as pd
import awswrangler as wr

# AWS region + session
region = "us-east-1"
session = boto3.Session(region_name=region)

# S3 bucket + prefixes
bucket = "aai540-group5-public-866792937762-us-east-1-an"
raw_prefix = "sdwa/raw/"
parquet_prefix = "sdwa/parquet/"

# Athena database
database = "sdwa_db"

# SDWA raw files
sdwa_files = {
    "events_milestones": "SDWA_EVENTS_MILESTONES.csv",
    "violations": "SDWA_VIOLATIONS_ENFORCEMENT.csv",
    "pws": "SDWA_PUB_WATER_SYSTEMS.csv",
    "lcr": "SDWA_LCR_SAMPLES.csv",
    "facilities": "SDWA_FACILITIES.csv",
    "service_areas": "SDWA_SERVICE_AREAS.csv",
    "geographic_areas": "SDWA_GEOGRAPHIC_AREAS.csv",
    "ref_codes": "SDWA_REF_CODE_VALUES.csv",
    "ansi_areas": "SDWA_REF_ANSI_AREAS.csv",
    "pn_assoc": "SDWA_PN_VIOLATION_ASSOC.csv",
    "site_visits": "SDWA_SITE_VISITS.csv",
}

# -----
# Utility: S3 object size
# -----
```

```

def get_s3_size_mb(key: str) -> float:
    s3 = session.client("s3")
    resp = s3.head_object(Bucket=bucket, Key=f"{raw_prefix}{key}")
    return resp["ContentLength"] / (1024 * 1024)

# -----
# Helper: read CSV as all STRING
# -----
def _read_csv_all_string(key: str, nrows=None):
    path = f"s3://{bucket}/{raw_prefix}{key}"
    df = wr.s3.read_csv(path, dtype=str, low_memory=False, nrows=nrows)
    return df.astype(str)

# -----
# Helper: write Parquet as all STRING
# -----
def _write_parquet_all_string(table_name: str, df: pd.DataFrame):
    out_path = f"s3://{bucket}/{parquet_prefix}{table_name}/"
    wr.s3.to_parquet(df=df, path=out_path, dataset=True, mode="overwrite")
    print(f"Parquet written: {out_path}")

```

Collecting sagemaker==2.218.0

Using cached sagemaker-2.218.0-py3-none-any.whl.metadata (14 kB)

Using cached sagemaker-2.218.0-py3-none-any.whl (1.5 MB)

Installing collected packages: sagemaker

Successfully installed sagemaker-2.218.0

Note: you may need to restart the kernel to use updated packages.

Collecting ppft

Using cached ppft-1.7.8-py3-none-any.whl.metadata (12 kB)

Using cached ppft-1.7.8-py3-none-any.whl (56 kB)

Installing collected packages: ppft

Successfully installed ppft-1.7.8

Note: you may need to restart the kernel to use updated packages.

Collecting pathos

Using cached pathos-0.3.5-py3-none-any.whl.metadata (11 kB)

Using cached pathos-0.3.5-py3-none-any.whl (82 kB)

Installing collected packages: pathos

Successfully installed pathos-0.3.5

Note: you may need to restart the kernel to use updated packages.

2. SDWA Ingestion Pipeline (Normal + Streaming Modes)

This section implements the complete ingestion workflow for all SDWA tables. Files smaller than 500 MB are ingested normally, while large files are streamed in chunks to avoid memory exhaustion. All outputs are written to the S3 datalake in Parquet format.

```

In [2]: # =====
# 2. SDWA Ingestion Pipeline (Normal + Streaming Modes)
# =====
# This cell controls HOW each SDWA CSV file is ingested from S3.
# The pipeline automatically chooses between:
# - NORMAL ingestion (load entire CSV into memory)

```

```

# - STREAMING ingestion (Load CSV in chunks to avoid memory issues)
#
# Why this matters:
# Some SDWA files are tiny (2-20 MB), while others are massive (3-4 GB).
# Loading a 4 GB CSV into memory will crash SageMaker Studio.
# Therefore, we dynamically choose the safest ingestion method per file.
#
# All ingestion paths convert every column to STRING to avoid schema drift.

def ingest_normal_csv(table_name: str, filename: str):
    """
    NORMAL INGESTION PATH
    -----

    This function is used for small and medium-sized CSV files that can safely
    fit into memory. It loads the entire CSV at once, converts all columns to
    STRING, and writes the result to S3 as a Parquet dataset.

    Parameters:
        table_name (str): Logical SDWA table name (e.g., 'violations')
        filename (str): Raw CSV filename in S3 (e.g., 'SDWA_VIOLATIONS_ENFORCEMENT.
    """

    print(f"\n=== NORMAL INGESTION: {table_name} ({filename}) ===")
    print(f"Reading CSV from: s3://{bucket}/{raw_prefix}/{filename}")

    # Load entire CSV into memory (safe only for smaller files)
    df = _read_csv_all_string(filename)

    print(f"Loaded {len(df):,} rows for '{table_name}'")

    # Write the fully Loaded DataFrame to S3 as Parquet
    _write_parquet_all_string(table_name, df)

    print(f"Completed NORMAL ingestion for '{table_name}'")

def ingest_large_csv_streaming(table_name: str, filename: str, chunksize=200_000):
    """
    STREAMING INGESTION PATH
    -----

    This function is used for large CSV files (hundreds of MB to multiple GB).
    Instead of loading the entire file at once, we read it in fixed-size chunks.
    Each chunk is converted to STRING and appended to a Parquet dataset in S3.

    Why streaming?
        - Prevents memory exhaustion in SageMaker Studio
        - Allows ingestion of multi-GB files safely
        - Produces the same final Parquet dataset as normal ingestion

    Parameters:
        table_name (str): Logical SDWA table name
        filename (str): Raw CSV filename
        chunksize (int): Number of rows per chunk (default: 200,000)
    """

```

```

# Build full S3 paths for reading and writing
path = f"s3://{bucket}/{raw_prefix}{filename}"
out_path = f"s3://{bucket}/{parquet_prefix}{table_name}/"

print(f"\n=== STREAMING INGESTION: {table_name} ({filename}) ===")
print(f"Streaming CSV from: {path}")
print(f"Writing Parquet chunks to: {out_path}")

first = True          # Controls overwrite vs append mode
total_rows = 0        # Running total of rows processed
chunk_idx = 0         # Chunk counter for logging

# Read CSV in chunks using AWS Wrangler
for chunk in wr.s3.read_csv(path, dtype=str, low_memory=False, chunksize=chunks

    chunk_idx += 1
    chunk = chunk.astype(str) # Ensure all columns remain STRING
    rows = len(chunk)
    total_rows += rows

    # First chunk overwrites any existing data; subsequent chunks append
    mode = "overwrite" if first else "append"
    first = False

    # Write chunk to Parquet dataset
    wr.s3.to_parquet(df=chunk, path=out_path, dataset=True, mode=mode)

    print(f"[chunk {chunk_idx}] wrote {rows:,} rows (mode={mode})")

print(f"Completed STREAMING ingestion for '{table_name}' ({total_rows:,} total

def ingest_table(table_name: str, filename: str, threshold_mb=500.0):
    """
    INGESTION ROUTER
    -----
    This function decides whether to use NORMAL or STREAMING ingestion
    based on the file size in S3.

    Logic:
    - If file size > threshold (default 500 MB) → STREAMING
    - Otherwise → NORMAL

    Why 500 MB?
    - Safe upper bound for loading CSVs into memory in SageMaker Studio
    - Prevents kernel crashes and OOM errors
    """

    # Determine file size in MB
    size = get_s3_size_mb(filename)

    print(f"\n=====")
    print(f"INGESTING TABLE: {table_name}")
    print(f"File: {filename}")
    print(f"Size: {size:.2f} MB (threshold: {threshold_mb} MB)")
    print(f"=====")

```

```

# Route to correct ingestion method
if size > threshold_mb:
    print(f"Using STREAMING ingestion for '{table_name}'")
    ingest_large_csv_streaming(table_name, filename)
else:
    print(f"Using NORMAL ingestion for '{table_name}'")
    ingest_normal_csv(table_name, filename)

print(f"Completed ingestion for '{table_name}'")

# -----
# Execute ingestion for ALL SDWA tables defined earlier
# -----
for key, filename in sdwa_files.items():
    ingest_table(key, filename)

```

```

=====
INGESTING TABLE: events_milestones
File: SDWA_EVENTS_MILESTONES.csv
Size: 65.76 MB (threshold: 500.0 MB)
=====
Using NORMAL ingestion for 'events_milestones'

```

```

=== NORMAL INGESTION: events_milestones (SDWA_EVENTS_MILESTONES.csv) ===
Reading CSV from: s3://aai540-group5-public-866792937762-us-east-1-an/sdwa/raw/SDWA_
EVENTS_MILESTONES.csv

```

```

2026-06-03 22:54:37,241 WARNING services.py:2137 -- WARNING: The object store is using /tmp instead of /dev/shm because /dev/shm has only 1908383744 bytes available. This will harm performance! You may be able to free up space by deleting files in /dev/shm. If you are inside a Docker container, you can increase /dev/shm size by passing '--shm-size=4.61gb' to 'docker run' (or add it to the run_options list in a Ray cluster config). Make sure to set this to more than 30% of available RAM.
2026-06-03 22:54:37,376 INFO worker.py:2007 -- Started a local Ray instance.
/opt/conda/lib/python3.12/site-packages/ray/_private/worker.py:2046: FutureWarning: Tip: In future versions of Ray, Ray will no longer override accelerator visible devices env var if num_gpus=0 or num_gpus=None (default). To enable this behavior and turn off this error message, set RAY_ACCEL_ENV_VAR_OVERRIDE_ON_ZERO=0
  warnings.warn(

```

Loaded 360,370 rows for 'events_milestones'
Parquet written: s3://aai540-group5-public-866792937762-us-east-1-an/sdwa/parquet/ev
ents_milestones/
Completed NORMAL ingestion for 'events_milestones'
Completed ingestion for 'events_milestones'

=====
INGESTING TABLE: violations
File: SDWA_VIOLATIONS_ENFORCEMENT.csv
Size: 3886.44 MB (threshold: 500.0 MB)
=====
Using STREAMING ingestion for 'violations'

=== STREAMING INGESTION: violations (SDWA_VIOLATIONS_ENFORCEMENT.csv) ===
Streaming CSV from: s3://aai540-group5-public-866792937762-us-east-1-an/sdwa/raw/SDW
A_VIOLATIONS_ENFORCEMENT.csv
Writing Parquet chunks to: s3://aai540-group5-public-866792937762-us-east-1-an/sdwa/
parquet/violations/
[chunk 1] wrote 200,000 rows (mode=overwrite)
[chunk 2] wrote 200,000 rows (mode=append)
[chunk 3] wrote 200,000 rows (mode=append)
[chunk 4] wrote 200,000 rows (mode=append)
[chunk 5] wrote 200,000 rows (mode=append)
[chunk 6] wrote 200,000 rows (mode=append)
[chunk 7] wrote 200,000 rows (mode=append)
[chunk 8] wrote 200,000 rows (mode=append)
[chunk 9] wrote 200,000 rows (mode=append)
[chunk 10] wrote 200,000 rows (mode=append)
[chunk 11] wrote 200,000 rows (mode=append)
[chunk 12] wrote 200,000 rows (mode=append)
[chunk 13] wrote 200,000 rows (mode=append)
[chunk 14] wrote 200,000 rows (mode=append)
[chunk 15] wrote 200,000 rows (mode=append)
[chunk 16] wrote 200,000 rows (mode=append)
[chunk 17] wrote 200,000 rows (mode=append)
[chunk 18] wrote 200,000 rows (mode=append)
[chunk 19] wrote 200,000 rows (mode=append)
[chunk 20] wrote 200,000 rows (mode=append)
[chunk 21] wrote 200,000 rows (mode=append)
[chunk 22] wrote 200,000 rows (mode=append)
[chunk 23] wrote 200,000 rows (mode=append)
[chunk 24] wrote 200,000 rows (mode=append)
[chunk 25] wrote 200,000 rows (mode=append)
[chunk 26] wrote 200,000 rows (mode=append)
[chunk 27] wrote 200,000 rows (mode=append)
[chunk 28] wrote 200,000 rows (mode=append)
[chunk 29] wrote 200,000 rows (mode=append)
[chunk 30] wrote 200,000 rows (mode=append)
[chunk 31] wrote 200,000 rows (mode=append)
[chunk 32] wrote 200,000 rows (mode=append)
[chunk 33] wrote 200,000 rows (mode=append)
[chunk 34] wrote 200,000 rows (mode=append)
[chunk 35] wrote 200,000 rows (mode=append)
[chunk 36] wrote 200,000 rows (mode=append)
[chunk 37] wrote 200,000 rows (mode=append)
[chunk 38] wrote 200,000 rows (mode=append)

```
[chunk 39] wrote 200,000 rows (mode=append)
[chunk 40] wrote 200,000 rows (mode=append)
[chunk 41] wrote 200,000 rows (mode=append)
[chunk 42] wrote 200,000 rows (mode=append)
[chunk 43] wrote 200,000 rows (mode=append)
[chunk 44] wrote 200,000 rows (mode=append)
[chunk 45] wrote 200,000 rows (mode=append)
[chunk 46] wrote 200,000 rows (mode=append)
[chunk 47] wrote 200,000 rows (mode=append)
[chunk 48] wrote 200,000 rows (mode=append)
[chunk 49] wrote 200,000 rows (mode=append)
[chunk 50] wrote 200,000 rows (mode=append)
[chunk 51] wrote 200,000 rows (mode=append)
[chunk 52] wrote 200,000 rows (mode=append)
[chunk 53] wrote 200,000 rows (mode=append)
[chunk 54] wrote 200,000 rows (mode=append)
[chunk 55] wrote 200,000 rows (mode=append)
[chunk 56] wrote 200,000 rows (mode=append)
[chunk 57] wrote 200,000 rows (mode=append)
[chunk 58] wrote 200,000 rows (mode=append)
[chunk 59] wrote 200,000 rows (mode=append)
[chunk 60] wrote 200,000 rows (mode=append)
[chunk 61] wrote 200,000 rows (mode=append)
[chunk 62] wrote 200,000 rows (mode=append)
[chunk 63] wrote 200,000 rows (mode=append)
[chunk 64] wrote 200,000 rows (mode=append)
[chunk 65] wrote 200,000 rows (mode=append)
[chunk 66] wrote 200,000 rows (mode=append)
[chunk 67] wrote 200,000 rows (mode=append)
[chunk 68] wrote 200,000 rows (mode=append)
[chunk 69] wrote 200,000 rows (mode=append)
[chunk 70] wrote 200,000 rows (mode=append)
[chunk 71] wrote 200,000 rows (mode=append)
[chunk 72] wrote 200,000 rows (mode=append)
[chunk 73] wrote 200,000 rows (mode=append)
[chunk 74] wrote 200,000 rows (mode=append)
[chunk 75] wrote 200,000 rows (mode=append)
[chunk 76] wrote 200,000 rows (mode=append)
[chunk 77] wrote 98,031 rows (mode=append)
Completed STREAMING ingestion for 'violations' (15,298,031 total rows)
Completed ingestion for 'violations'
```

```
=====
```

```
INGESTING TABLE: pws
```

```
File: SDWA_PUB_WATER_SYSTEMS.csv
```

```
Size: 123.12 MB (threshold: 500.0 MB)
```

```
=====
```

```
Using NORMAL ingestion for 'pws'
```

```
=== NORMAL INGESTION: pws (SDWA_PUB_WATER_SYSTEMS.csv) ===
```

```
Reading CSV from: s3://aai540-group5-public-866792937762-us-east-1-an/sdwa/raw/SDWA_
PUB_WATER_SYSTEMS.csv
```

```
Loaded 433,698 rows for 'pws'
```

```
Parquet written: s3://aai540-group5-public-866792937762-us-east-1-an/sdwa/parquet/pw
s/
```

```
Completed NORMAL ingestion for 'pws'
```

Completed ingestion for 'pws'

=====

INGESTING TABLE: lcr

File: SDWA_LCR_SAMPLES.csv

Size: 118.26 MB (threshold: 500.0 MB)

=====

Using NORMAL ingestion for 'lcr'

=== NORMAL INGESTION: lcr (SDWA_LCR_SAMPLES.csv) ===

Reading CSV from: s3://aai540-group5-public-866792937762-us-east-1-an/sdwa/raw/SDWA_LCR_SAMPLES.csv

Loaded 924,498 rows for 'lcr'

Parquet written: s3://aai540-group5-public-866792937762-us-east-1-an/sdwa/parquet/lcr/

Completed NORMAL ingestion for 'lcr'

Completed ingestion for 'lcr'

=====

INGESTING TABLE: facilities

File: SDWA_FACILITIES.csv

Size: 159.23 MB (threshold: 500.0 MB)

=====

Using NORMAL ingestion for 'facilities'

=== NORMAL INGESTION: facilities (SDWA_FACILITIES.csv) ===

Reading CSV from: s3://aai540-group5-public-866792937762-us-east-1-an/sdwa/raw/SDWA_FACILITIES.csv

Loaded 1,550,159 rows for 'facilities'

Parquet written: s3://aai540-group5-public-866792937762-us-east-1-an/sdwa/parquet/facilities/

Completed NORMAL ingestion for 'facilities'

Completed ingestion for 'facilities'

=====

INGESTING TABLE: service_areas

File: SDWA_SERVICE_AREAS.csv

Size: 18.42 MB (threshold: 500.0 MB)

=====

Using NORMAL ingestion for 'service_areas'

=== NORMAL INGESTION: service_areas (SDWA_SERVICE_AREAS.csv) ===

Reading CSV from: s3://aai540-group5-public-866792937762-us-east-1-an/sdwa/raw/SDWA_SERVICE_AREAS.csv

Loaded 422,099 rows for 'service_areas'

Parquet written: s3://aai540-group5-public-866792937762-us-east-1-an/sdwa/parquet/service_areas/

Completed NORMAL ingestion for 'service_areas'

Completed ingestion for 'service_areas'

=====

INGESTING TABLE: geographic_areas

File: SDWA_GEOGRAPHIC_AREAS.csv

Size: 38.84 MB (threshold: 500.0 MB)

=====

Using NORMAL ingestion for 'geographic_areas'


```
=== NORMAL INGESTION: geographic_areas (SDWA_GEOGRAPHIC_AREAS.csv) ===
Reading CSV from: s3://aai540-group5-public-866792937762-us-east-1-an/sdwa/raw/SDWA_
GEOGRAPHIC_AREAS.csv
Loaded 577,661 rows for 'geographic_areas'
Parquet written: s3://aai540-group5-public-866792937762-us-east-1-an/sdwa/parquet/ge
ographic_areas/
Completed NORMAL ingestion for 'geographic_areas'
Completed ingestion for 'geographic_areas'
```

```
=====
INGESTING TABLE: ref_codes
File: SDWA_REF_CODE_VALUES.csv
Size: 0.12 MB (threshold: 500.0 MB)
=====
Using NORMAL ingestion for 'ref_codes'
```

```
=== NORMAL INGESTION: ref_codes (SDWA_REF_CODE_VALUES.csv) ===
Reading CSV from: s3://aai540-group5-public-866792937762-us-east-1-an/sdwa/raw/SDWA_
REF_CODE_VALUES.csv
Loaded 2,376 rows for 'ref_codes'
Parquet written: s3://aai540-group5-public-866792937762-us-east-1-an/sdwa/parquet/re
f_codes/
Completed NORMAL ingestion for 'ref_codes'
Completed ingestion for 'ref_codes'
```

```
=====
INGESTING TABLE: ansi_areas
File: SDWA_REF_ANSI_AREAS.csv
Size: 0.08 MB (threshold: 500.0 MB)
=====
Using NORMAL ingestion for 'ansi_areas'
```

```
=== NORMAL INGESTION: ansi_areas (SDWA_REF_ANSI_AREAS.csv) ===
Reading CSV from: s3://aai540-group5-public-866792937762-us-east-1-an/sdwa/raw/SDWA_
REF_ANSI_AREAS.csv
Loaded 3,235 rows for 'ansi_areas'
Parquet written: s3://aai540-group5-public-866792937762-us-east-1-an/sdwa/parquet/an
si_areas/
Completed NORMAL ingestion for 'ansi_areas'
Completed ingestion for 'ansi_areas'
```

```
=====
INGESTING TABLE: pn_assoc
File: SDWA_PN_VIOLATION_ASSOC.csv
Size: 45.99 MB (threshold: 500.0 MB)
=====
Using NORMAL ingestion for 'pn_assoc'
```

```
=== NORMAL INGESTION: pn_assoc (SDWA_PN_VIOLATION_ASSOC.csv) ===
Reading CSV from: s3://aai540-group5-public-866792937762-us-east-1-an/sdwa/raw/SDWA_
PN_VIOLATION_ASSOC.csv
Loaded 378,063 rows for 'pn_assoc'
Parquet written: s3://aai540-group5-public-866792937762-us-east-1-an/sdwa/parquet/pn
_assoc/
Completed NORMAL ingestion for 'pn_assoc'
```

Completed ingestion for 'pn_assoc'

=====

INGESTING TABLE: site_visits

File: SDWA_SITE_VISITS.csv

Size: 371.89 MB (threshold: 500.0 MB)

=====

Using NORMAL ingestion for 'site_visits'

=== NORMAL INGESTION: site_visits (SDWA_SITE_VISITS.csv) ===

Reading CSV from: s3://aai540-group5-public-866792937762-us-east-1-an/sdwa/raw/SDWA_SITE_VISITS.csv

Loaded 2,478,266 rows for 'site_visits'

Parquet written: s3://aai540-group5-public-866792937762-us-east-1-an/sdwa/parquet/site_visits/

Completed NORMAL ingestion for 'site_visits'

Completed ingestion for 'site_visits'

3. Athena Table Registration and Schema Validation

This section registers each Parquet dataset as an Athena table using a consistent all-STRING schema. A schema sanity check confirms that all tables were ingested correctly and are ready for SQL-based exploration and feature engineering.

```
In [3]: # =====
# 3. Athena Table Registration and Schema Validation
# =====
# PURPOSE OF THIS CELL:
# -----
# After ingestion, all SDWA datasets now exist in S3 as Parquet files.
# However, Athena cannot query raw S3 files unless they are REGISTERED
# in the AWS Glue Data Catalog as tables.
#
# This cell:
#   1. Ensures the Athena database exists
#   2. Registers each Parquet dataset as an Athena table
#   3. Forces ALL columns to STRING (critical for schema stability)
#   4. Performs a schema sanity check to confirm ingestion succeeded
#
# Why force STRING?
#   - SDWA CSVs contain inconsistent typing (numbers stored as strings, etc.)
#   - Athena will break if partitions have mismatched types
#   - STRING is the safest, most flexible type for raw ingestion
#
# After this cell runs:
#   - You can query all SDWA tables using SQL in Athena
#   - Feature engineering (next cell) can use SQL safely

def safe_create_parquet_table(database: str, table: str, path: str):
    """
```

Registers a Parquet dataset in S3 as an Athena table.

This function:

- Reads ONE Parquet file to infer column names
- Assigns STRING type to every column
- Creates or overwrites the Athena table definition

Parameters:

database (str): Athena database name
table (str): Table name to create in Glue Catalog
path (str): S3 path to the Parquet dataset

"""

List all Parquet objects under the dataset path

objects = wr.s3.list_objects(path)

Read a single Parquet file to inspect the schema

sample = wr.s3.read_parquet(objects[0])

Build a dictionary mapping column_name → "string"

This ensures schema consistency across all partitions

columns_types = {col: "string" for col in sample.columns}

Register the table in the Glue Data Catalog

```
wr.catalog.create_parquet_table(
    database=database,
    table=table,
    path=path,
    columns_types=columns_types,
    mode="overwrite",  # Always overwrite to ensure schema consistency
)
```

print(f"Registered Athena table: {table}")

-----

Ensure the Athena database exists (idempotent)

-----

If the database already exists, nothing happens.

If it does not exist, it is created.

if database not in wr.catalog.databases()["Database"].values:

print(f"Creating Athena database '{database}'...")

wr.catalog.create_database(name=database)

else:

print(f"Athena database '{database}' already exists.")

-----

Register ALL SDWA tables as Athena tables

-----

for table_name in sdwa_files.keys():

Build the S3 path to the Parquet dataset for this table

path = f"s3://{bucket}/{parquet_prefix}{table_name}/"

print(f"\nRegistering table '{table_name}' from: {path}")

```

# Register the table using the helper function above
safe_create_parquet_table(database, table_name, path)

# -----
# Schema Sanity Check
# -----
# PURPOSE:
#   - Confirms that each Parquet dataset is readable
#   - Confirms that all columns are STRING
#   - Confirms that ingestion produced valid Parquet files
#
# We read ONLY ONE chunk from each dataset to avoid loading large files.

for table_name in sdwa_files.keys():

    path = f"s3://{bucket}/{parquet_prefix}{table_name}/"
    print(f"\n=== Schema check for '{table_name}' ===")

    # Read the first chunk of the Parquet dataset
    df_iter = wr.s3.read_parquet(path, dataset=True, chunked=True)
    df_sample = next(df_iter)

    # Print the dtypes – all should be 'object' (STRING)
    print(df_sample.dtypes)

```

Athena database 'sdwa_db' already exists.

Registering table 'events_milestones' from: s3://aai540-group5-public-866792937762-us-east-1-an/sdwa/parquet/events_milestones/
Registered Athena table: events_milestones

Registering table 'violations' from: s3://aai540-group5-public-866792937762-us-east-1-an/sdwa/parquet/violations/
Registered Athena table: violations

Registering table 'pws' from: s3://aai540-group5-public-866792937762-us-east-1-an/sdwa/parquet/pws/
Registered Athena table: pws

Registering table 'lcr' from: s3://aai540-group5-public-866792937762-us-east-1-an/sdwa/parquet/lcr/
Registered Athena table: lcr

Registering table 'facilities' from: s3://aai540-group5-public-866792937762-us-east-1-an/sdwa/parquet/facilities/
Registered Athena table: facilities

Registering table 'service_areas' from: s3://aai540-group5-public-866792937762-us-east-1-an/sdwa/parquet/service_areas/
Registered Athena table: service_areas

Registering table 'geographic_areas' from: s3://aai540-group5-public-866792937762-us-east-1-an/sdwa/parquet/geographic_areas/
Registered Athena table: geographic_areas

Registering table 'ref_codes' from: s3://aai540-group5-public-866792937762-us-east-1-an/sdwa/parquet/ref_codes/
Registered Athena table: ref_codes

Registering table 'ansi_areas' from: s3://aai540-group5-public-866792937762-us-east-1-an/sdwa/parquet/ansi_areas/
Registered Athena table: ansi_areas

Registering table 'pn_assoc' from: s3://aai540-group5-public-866792937762-us-east-1-an/sdwa/parquet/pn_assoc/
Registered Athena table: pn_assoc

Registering table 'site_visits' from: s3://aai540-group5-public-866792937762-us-east-1-an/sdwa/parquet/site_visits/
Registered Athena table: site_visits

=== Schema check for 'events_milestones' ===

SUBMISSIONYEARQUARTER	string[python]
PWSID	string[python]
EVENT_SCHEDULE_ID	string[python]
EVENT_END_DATE	string[python]
EVENT_ACTUAL_DATE	string[python]
EVENT_COMMENTS_TEXT	string[python]
EVENT_MILESTONE_CODE	string[python]
EVENT_REASON_CODE	string[python]
FIRST_REPORTED_DATE	string[python]

LAST_REPORTED_DATE string[python]
dtype: object

=== Schema check for 'violations' ===

SUBMISSIONYEARQUARTER	string[python]
PWSID	string[python]
VIOLATION_ID	string[python]
FACILITY_ID	string[python]
COMPL_PER_BEGIN_DATE	string[python]
COMPL_PER_END_DATE	string[python]
NON_COMPL_PER_BEGIN_DATE	string[python]
NON_COMPL_PER_END_DATE	string[python]
PWS_DEACTIVATION_DATE	string[python]
VIOLATION_CODE	string[python]
VIOLATION_CATEGORY_CODE	string[python]
IS_HEALTH_BASED_IND	string[python]
CONTAMINANT_CODE	string[python]
VIOL_MEASURE	string[python]
UNIT_OF_MEASURE	string[python]
FEDERAL_MCL	string[python]
STATE_MCL	string[python]
IS_MAJOR_VIOL_IND	string[python]
SEVERITY_IND_CNT	string[python]
CALCULATED_RTC_DATE	string[python]
VIOLATION_STATUS	string[python]
PUBLIC_NOTIFICATION_TIER	string[python]
CALCULATED_PUB_NOTIF_TIER	string[python]
VIOL_ORIGINATOR_CODE	string[python]
SAMPLE_RESULT_ID	string[python]
CORRECTIVE_ACTION_ID	string[python]
RULE_CODE	string[python]
RULE_GROUP_CODE	string[python]
RULE_FAMILY_CODE	string[python]
VIOL_FIRST_REPORTED_DATE	string[python]
VIOL_LAST_REPORTED_DATE	string[python]
ENFORCEMENT_ID	string[python]
ENFORCEMENT_DATE	string[python]
ENFORCEMENT_ACTION_TYPE_CODE	string[python]
ENF_ACTION_CATEGORY	string[python]
ENF_ORIGINATOR_CODE	string[python]
ENF_FIRST_REPORTED_DATE	string[python]
ENF_LAST_REPORTED_DATE	string[python]

dtype: object

=== Schema check for 'pws' ===

SUBMISSIONYEARQUARTER	string[python]
PWSID	string[python]
PWS_NAME	string[python]
PRIMACY_AGENCY_CODE	string[python]
EPA_REGION	string[python]
SEASON_BEGIN_DATE	string[python]
SEASON_END_DATE	string[python]
PWS_ACTIVITY_CODE	string[python]
PWS_DEACTIVATION_DATE	string[python]
PWS_TYPE_CODE	string[python]
DBPR_SCHEDULE_CAT_CODE	string[python]

CDS_ID	string[python]
GW_SW_CODE	string[python]
LT2_SCHEDULE_CAT_CODE	string[python]
OWNER_TYPE_CODE	string[python]
POPULATION_SERVED_COUNT	string[python]
POP_CAT_2_CODE	string[python]
POP_CAT_3_CODE	string[python]
POP_CAT_4_CODE	string[python]
POP_CAT_5_CODE	string[python]
POP_CAT_11_CODE	string[python]
PRIMACY_TYPE	string[python]
PRIMARY_SOURCE_CODE	string[python]
IS_GRANT_ELIGIBLE_IND	string[python]
IS_WHOLESALE_IND	string[python]
IS_SCHOOL_OR_DAYCARE_IND	string[python]
SERVICE_CONNECTIONS_COUNT	string[python]
SUBMISSION_STATUS_CODE	string[python]
ORG_NAME	string[python]
ADMIN_NAME	string[python]
EMAIL_ADDR	string[python]
PHONE_NUMBER	string[python]
PHONE_EXT_NUMBER	string[python]
FAX_NUMBER	string[python]
ALT_PHONE_NUMBER	string[python]
ADDRESS_LINE1	string[python]
ADDRESS_LINE2	string[python]
CITY_NAME	string[python]
ZIP_CODE	string[python]
COUNTRY_CODE	string[python]
FIRST_REPORTED_DATE	string[python]
LAST_REPORTED_DATE	string[python]
STATE_CODE	string[python]
SOURCE_WATER_PROTECTION_CODE	string[python]
SOURCE_PROTECTION_BEGIN_DATE	string[python]
OUTSTANDING_PERFORMER	string[python]
OUTSTANDING_PERFORM_BEGIN_DATE	string[python]
REDUCED_RTCT_MONITORING	string[python]
REDUCED_MONITORING_BEGIN_DATE	string[python]
REDUCED_MONITORING_END_DATE	string[python]
SEASONAL_STARTUP_SYSTEM	string[python]
dtype: object	

=== Schema check for 'lcr' ===

SUBMISSIONYEARQUARTER	string[python]
PWSID	string[python]
SAMPLE_ID	string[python]
SAMPLING_END_DATE	string[python]
SAMPLING_START_DATE	string[python]
RECONCILIATION_ID	string[python]
SAMPLE_FIRST_REPORTED_DATE	string[python]
SAMPLE_LAST_REPORTED_DATE	string[python]
SAR_ID	string[python]
CONTAMINANT_CODE	string[python]
RESULT_SIGN_CODE	string[python]
SAMPLE_MEASURE	string[python]
UNIT_OF_MEASURE	string[python]

```
SAR_FIRST_REPORTED_DATE      string[python]
SAR_LAST_REPORTED_DATE       string[python]
dtype: object
```

=== Schema check for 'facilities' ===

```
SUBMISSIONYEARQUARTER      string[python]
PWSID                      string[python]
FACILITY_ID                string[python]
FACILITY_NAME              string[python]
STATE_FACILITY_ID          string[python]
FACILITY_ACTIVITY_CODE      string[python]
FACILITY_DEACTIVATION_DATE  string[python]
FACILITY_TYPE_CODE         string[python]
SUBMISSION_STATUS_CODE      string[python]
IS_SOURCE_IND              string[python]
WATER_TYPE_CODE            string[python]
AVAILABILITY_CODE          string[python]
SELLER_TREATMENT_CODE       string[python]
SELLER_PWSID               string[python]
SELLER_PWS_NAME            string[python]
FILTRATION_STATUS_CODE      string[python]
IS_SOURCE_TREATED_IND       string[python]
FIRST_REPORTED_DATE         string[python]
LAST_REPORTED_DATE          string[python]
dtype: object
```

=== Schema check for 'service_areas' ===

```
SUBMISSIONYEARQUARTER      string[python]
PWSID                      string[python]
SERVICE_AREA_TYPE_CODE     string[python]
IS_PRIMARY_SERVICE_AREA_CODE string[python]
FIRST_REPORTED_DATE         string[python]
LAST_REPORTED_DATE          string[python]
dtype: object
```

=== Schema check for 'geographic_areas' ===

```
SUBMISSIONYEARQUARTER      string[python]
PWSID                      string[python]
GEO_ID                    string[python]
AREA_TYPE_CODE             string[python]
TRIBAL_CODE                string[python]
STATE_SERVED               string[python]
ANSI_ENTITY_CODE           string[python]
ZIP_CODE_SERVED            string[python]
CITY_SERVED                string[python]
COUNTY_SERVED             string[python]
LAST_REPORTED_DATE          string[python]
dtype: object
```

=== Schema check for 'ref_codes' ===

```
VALUE_TYPE                 string[python]
VALUE_CODE                  string[python]
VALUE_DESCRIPTION           string[python]
dtype: object
```

=== Schema check for 'ansi_areas' ===


```

ANSI_STATE_CODE      string[python]
ANSI_ENTITY_CODE      string[python]
ANSI_NAME             string[python]
STATE_CODE            string[python]
dtype: object

```

```

=== Schema check for 'pn_assoc' ===
SUBMISSIONYEARQUARTER    string[python]
PWSID                    string[python]
PN_VIOLATION_ID          string[python]
RELATED_VIOLATION_ID     string[python]
COMPL_PER_BEGIN_DATE     string[python]
COMPL_PER_END_DATE       string[python]
NON_COMPL_PER_BEGIN_DATE string[python]
NON_COMPL_PER_END_DATE   string[python]
VIOLATION_CODE           string[python]
CONTAMINANT_CODE         string[python]
FIRST_REPORTED_DATE      string[python]
LAST_REPORTED_DATE       string[python]
dtype: object

```

```

=== Schema check for 'site_visits' ===
SUBMISSIONYEARQUARTER    string[python]
PWSID                    string[python]
VISIT_ID                 string[python]
VISIT_DATE               string[python]
AGENCY_TYPE_CODE         string[python]
VISIT_REASON_CODE        string[python]
MANAGEMENT_OPS_EVAL_CODE string[python]
SOURCE_WATER_EVAL_CODE   string[python]
SECURITY_EVAL_CODE       string[python]
PUMPS_EVAL_CODE          string[python]
OTHER_EVAL_CODE          string[python]
COMPLIANCE_EVAL_CODE     string[python]
DATA_VERIFICATION_EVAL_CODE string[python]
TREATMENT_EVAL_CODE      string[python]
FINISHED_WATER_STOR_EVAL_CODE string[python]
DISTRIBUTION_EVAL_CODE   string[python]
FINANCIAL_EVAL_CODE      string[python]
VISIT_COMMENTS           string[python]
FIRST_REPORTED_DATE      string[python]
LAST_REPORTED_DATE       string[python]
dtype: object

```

4. Exploratory Data Analysis (EDA)

This section performs initial exploration of the SDWA datasets using Athena queries and summary statistics. The goal is to understand dataset size, structure, and key distributions before performing feature engineering.

```

In [4]: # =====
# 4. Exploratory Data Analysis (EDA) – Memory-Safe Version
# =====

```

```

# PURPOSE OF THIS CELL:
# -----
# This cell performs EDA (Exploratory Data Analysis) on the SDWA datasets.
# The original version loaded entire Parquet tables into memory, which is unsafe
# for large datasets (e.g., violations, LCR).
#
# This version is 100% memory-safe because:
#   • ALL row counts use Athena SQL (COUNT(*))
#   • ALL summaries use Athena SQL
#   • NO full-table DataFrame loads occur
#
# Athena handles the heavy lifting, and we only retrieve small aggregated results.

# -----
# 1. Row counts for each SDWA table (memory-safe)
# -----
# WHY THIS MATTERS:
#   • Confirms ingestion succeeded
#   • Helps understand dataset scale
#   • Avoids loading multi-GB Parquet files into memory
#
# We use Athena COUNT(*) instead of wr.s3.read_parquet().

print("=== ROW COUNTS FOR ALL SDWA TABLES (Memory-Safe) ===")

for table in sdwa_files.keys():

    query = f"SELECT COUNT(*) AS cnt FROM {table}"

    result = wr.athena.read_sql_query(query, database=database)
    count = int(result['cnt'][0])

    print(f"{table:20s}: {count:,} rows")

# -----
# 2. Violations summary by violation category
# -----
# WHY THIS MATTERS:
#   • Shows distribution of violation types
#   • Helps validate ingestion and schema
#   • Useful for feature engineering
#
# Athena performs the aggregation – memory-safe.

print("\n=== VIOLATIONS BY CATEGORY ===")

viol_summary = wr.athena.read_sql_query(
    """
    SELECT violation_category_code,
           COUNT(*) AS count
    FROM violations
    GROUP BY violation_category_code
    ORDER BY count DESC
    """,

```

```

        database=database,
    )

display(viol_summary)

# -----
# 3. LCR contaminant distribution
# -----
# WHY THIS MATTERS:
#   • Identifies contaminants present in LCR samples
#   • Helps validate sample_measure and contaminant_code fields
#   • Useful for modeling and feature engineering
#
# Again, Athena handles the aggregation.

print("\n=== LCR CONTAMINANT DISTRIBUTION ===")

lcr_summary = wr.athena.read_sql_query(
    """
    SELECT contaminant_code,
           COUNT(*) AS count
    FROM lcr
    GROUP BY contaminant_code
    ORDER BY count DESC
    """,
    database=database,
)

display(lcr_summary)

# -----
# 4. Population served statistics (memory-safe)
# -----
# WHY THIS MATTERS:
#   • population_served_count is a key modeling feature
#   • Helps identify outliers (very large or very small systems)
#   • Confirms numeric casting works correctly
#
# Athena performs MIN/AVG/MAX – no DataFrame Loads.

print("\n=== POPULATION SERVED SUMMARY ===")

pws_stats = wr.athena.read_sql_query(
    """
    SELECT
        MIN(CAST(population_served_count AS DOUBLE)) AS min_pop,
        AVG(CAST(population_served_count AS DOUBLE)) AS avg_pop,
        MAX(CAST(population_served_count AS DOUBLE)) AS max_pop
    FROM pws
    """,
    database=database,
)

display(pws_stats)

```

```

# -----
# 5. PWS count by state (memory-safe)
# -----
# WHY THIS MATTERS:
#   • Shows geographic distribution of water systems
#   • Helps validate state_code field
#   • Useful for understanding dataset coverage

print("\n=== PWS COUNT BY STATE ===")

state_summary = wr.athena.read_sql_query(
    """
    SELECT state_code,
           COUNT(*) AS pws_count
    FROM pws
    GROUP BY state_code
    ORDER BY pws_count DESC
    """,
    database=database,
)

display(state_summary)

print("\nMemory-safe EDA complete.")

```

=== ROW COUNTS FOR ALL SDWA TABLES (Memory-Safe) ===

events_milestones	: 360,370 rows
violations	: 15,298,031 rows
pws	: 433,698 rows
lcr	: 924,498 rows
facilities	: 1,550,159 rows
service_areas	: 422,099 rows
geographic_areas	: 577,661 rows
ref_codes	: 2,376 rows
ansi_areas	: 3,235 rows
pn_assoc	: 378,063 rows
site_visits	: 2,478,266 rows

=== VIOLATIONS BY CATEGORY ===

	violation_category_code	count
0	MR	10566524
1	MCL	1637907
2	nan	996827
3	Other	937358
4	MON	716579
5	TT	340199
6	RPT	101368
7	MRDL	1269

=== LCR CONTAMINANT DISTRIBUTION ===

	contaminant_code	count
0	PB90	885923
1	CU90	38575

=== POPULATION SERVED SUMMARY ===

	min_pop	avg_pop	max_pop
0	0.0	1076.670234	9000000.0

=== PWS COUNT BY STATE ===

	state_code	pws_count
0	MI	26703
1	IL	25921
2	NY	24700
3	WI	23977
4	NC	23899
...	...	...
60	BC	3
61	PQ	2
62	AB	2
63	AP	1
64	NS	1

65 rows × 2 columns

Memory-safe EDA complete.

SDWA Data Dictionary Overview

This project uses the EPA Safe Drinking Water Act (SDWA) datasets contained in the provided ZIP archive. The following summarizes each file and its relevance to ingestion, feature engineering, and modeling.

Core SDWA Files

1. SDWA_EVENTS_MILESTONES.csv

Regulatory events, compliance milestones, and enforcement timelines.

2. SDWA_FACILITIES.csv

Facility-level metadata for public water systems, including facility type, ownership, water source, and population served.

3. SDWA_GEOGRAPHIC_AREAS.csv

Geographic mappings for systems, including county and state associations.

4. SDWA_LCR_SAMPLES.csv

Lead and Copper Rule sampling results used for contaminant-level features.

5. SDWA_PN_VIOLATION_ASSOC.csv

Associations between public notices and violations, supporting severity and communication tracking.

6. SDWA_PUB_WATER_SYSTEMS.csv

Master table of all Public Water Systems (PWS), including system metadata, population served, and system type.

7. SDWA_REF_ANSI_AREAS.csv

Reference table for geographic codes used in joins.

8. SDWA_REF_CODE_VALUES.csv

Reference table for violation codes, contaminant codes, and other categorical mappings.

9. SDWA_SERVICE_AREAS.csv

Service area boundaries and ZIP-to-PWSID relationships.

10. SDWA_SITE_VISITS.csv

Inspection and site visit records used for compliance and enforcement context.

11. SDWA_VIOLATIONS_ENFORCEM.csv

Primary violations dataset containing health-based violations, monitoring violations, enforcement actions, and compliance status.

Mapping to ML Feature Requirements

ML Feature	Source File(s)
Total violations	SDWA_VIOLATIONS_ENFORCEM
Health-based violations	SDWA_VIOLATIONS_ENFORCEM
Monitoring violations	SDWA_VIOLATIONS_ENFORCEM
Enforcement actions	SDWA_VIOLATIONS_ENFORCEM
Lead/copper levels	SDWA_LCR_SAMPLES
Population served	SDWA_PUB_WATER_SYSTEMS
Water source type	SDWA_FACILITIES, SDWA_PUB_WATER_SYSTEMS
ZIP code mapping	SDWA_SERVICE_AREAS, SDWA_GEOGRAPHIC_AREAS
Contaminant codes	SDWA_REF_CODE_VALUES

This header summarizes the SDWA dataset contents and their relevance to the Week-3 ingestion and feature engineering pipeline.

5. Feature Engineering at the PWS × Quarter Grain

This section constructs engineered features by aggregating violations, LCR samples, and PWS attributes at the grain:

PWSID × SUBMISSIONYEARQUARTER. This grain aligns with SDWA reporting structure and supports downstream modeling and monitoring tasks.

PWS (Public Water System)

A Public Water System is any water system that provides water for human consumption to at

least 15 service connections or regularly serves at least 25 people for at least 60 days per year, as defined by the U.S. Safe Drinking Water Act (SDWA). Each system is uniquely identified by a PWSID.

```
In [ ]: # =====
# 5. Feature Engineering (PWS × Quarter Grain)
# =====
# PURPOSE OF THIS CELL:
# -----
# Week 3 requires us to engineer features from the raw SDWA datasets.
# The correct grain for SDWA modeling is:
#
#     PWSID × SUBMISSIONYEARQUARTER
#
# Why this grain?
#   • Violations are reported quarterly
#   • LCR samples are reported quarterly
#   • PWS attributes are static (system-level)
#   • This grain aligns with EPA reporting structure
#
# In this cell, we:
#   1. Aggregate violations by PWS and quarter
#   2. Aggregate LCR samples by PWS and quarter
#   3. Pull static PWS attributes
#   4. Merge everything into a single engineered feature table
#
# This table becomes the foundation for:
#   • Week 4 model training
#   • Feature Store ingestion
#   • Train/val/test/prod splits

# -----
# 1. Violations features (quarterly)
# -----
# We compute:
#   • total_violations
#   • health_based_violations
#   • major_violations
#
# LOWER() is used to normalize PWSID and SUBMISSIONYEARQUARTER
# because SDWA data sometimes mixes uppercase/lowercase.

viol_sql = """
SELECT
    LOWER(pwsid) AS pwsid,
    LOWER(submissionyearquarter) AS submissionyearquarter,

    -- Count all violations for this PWS + quarter
    COUNT(*) AS total_violations,

    -- Count violations marked as health-based
    SUM(CASE WHEN is_health_based_ind = 'Y' THEN 1 ELSE 0 END) AS health_based_viol,

    -- Count violations marked as major
```



```

SUM(CASE WHEN is_major_viol_ind = 'Y' THEN 1 ELSE 0 END) AS major_violations

FROM violations
GROUP BY 1, 2
"""

print("Running violations feature query...")
viol_df = wr.athena.read_sql_query(viol_sql, database=database)
print(f"Violations features loaded: {len(viol_df):,} rows")

# -----
# 2. LCR sample features (quarterly)
# -----
# LCR = Lead and Copper Rule
# We compute:
#   • lcr_sample_count
#   • lcr_max_measure
#   • lcr_mean_measure
#
# SAMPLE_MEASURE is stored as STRING in Athena, so we CAST to DOUBLE.

lcr_sql = """
SELECT
    LOWER(pwsid) AS pwsid,
    LOWER(submissionyearquarter) AS submissionyearquarter,

    COUNT(*) AS lcr_sample_count,

    -- Maximum contaminant measurement for the quarter
    MAX(CAST(sample_measure AS DOUBLE)) AS lcr_max_measure,

    -- Average contaminant measurement for the quarter
    AVG(CAST(sample_measure AS DOUBLE)) AS lcr_mean_measure

FROM lcr
GROUP BY 1, 2
"""

print("Running LCR feature query...")
lcr_df = wr.athena.read_sql_query(lcr_sql, database=database)
print(f"LCR features loaded: {len(lcr_df):,} rows")

# -----
# 3. PWS static features (system-level)
# -----
# These features do NOT vary by quarter.
# They describe the water system itself:
#   • population_served
#   • pws_type_code
#   • owner_type_code
#   • state_code
#   • epa_region
#
# These will be merged onto every quarter for that PWS.

```

```

pws_sql = """
SELECT
    LOWER(pwsid) AS pwsid,

    -- Convert population to numeric
    CAST(population_served_count AS DOUBLE) AS population_served,

    pws_type_code,
    owner_type_code,
    state_code,
    epa_region

FROM pws
"""

print("Running PWS static feature query...")
pws_df = wr.athena.read_sql_query(pws_sql, database=database)
print(f"PWS static features loaded: {len(pws_df):,} rows")

# -----
# 4. Merge all feature sets into a single engineered table
# -----
# Merge order:
# violations → lcr → pws
#
# Merge keys:
# ["pwsid", "submissionyearquarter"]
#
# Why Lowercase?
# Because Athena returns lowercase aliases from LOWER()

print("\nMerging violations + LCR features...")
feat_df = viol_df.merge(
    lcr_df,
    on=["pwsid", "submissionyearquarter"],
    how="left" # Keep all violation rows even if LCR is missing
)

print("Merging PWS static features...")
feat_df = feat_df.merge(
    pws_df,
    on="pwsid",
    how="left" # Keep all rows even if PWS attributes missing
)

print(f"Final engineered feature table shape: {feat_df.shape}")

# -----
# 5. Handle missing numeric values
# -----
# Why?
# • Athena returns NULL for missing values
# • ML models cannot handle NaN values

```

```

# • We replace missing numeric values with 0.0
#
# This is safe because:
# • Missing violations = 0 violations
# • Missing LCR samples = no samples
# • Missing population_served = unknown (0 is placeholder)

numeric_cols = [
    "total_violations",
    "health_based_violations",
    "major_violations",
    "lcr_sample_count",
    "lcr_max_measure",
    "lcr_mean_measure",
    "population_served",
]

print("\nFilling missing numeric values with 0.0...")
for col in numeric_cols:
    feat_df[col] = feat_df[col].fillna(0.0)

# -----
# 6. Example derived feature
# -----
# Violations per 1,000 population
# This normalizes violations by system size.

print("Creating derived feature: violations_per_1k_pop...")
feat_df["violations_per_1k_pop"] = feat_df.apply(
    lambda r: (r["total_violations"] / r["population_served"] * 1000.0)
    if r["population_served"] and r["population_served"] > 0
    else 0.0,
    axis=1,
)

print("\nFeature engineering complete.")
feat_df.head()

```

6. Persist Engineered Features to S3 and Athena

This section writes the engineered feature table to the S3 datalake and registers it as an Athena table. This creates a reproducible, queryable feature artifact for Week-4 modeling.

```

In [ ]: # =====
# 6. Persist Engineered Features to S3 and Athena
# =====
# PURPOSE OF THIS CELL:
# -----
# At this point, we have a fully engineered feature table (feat_df)
# containing quarterly violation metrics, LCR metrics, and PWS attributes.

```

```

#
# Now we must:
# 1. Save the engineered features to S3 in Parquet format
# 2. Register the engineered feature table in Athena
#
# WHY THIS IS REQUIRED:
# • Week 3 deliverables require storing engineered features
# • Week 4 model training will read features from S3/Athena
# • Parquet is efficient, compressed, and columnar (best for analytics)
# • Athena requires a Glue Catalog table to query the dataset
#
# After this cell runs:
# • The engineered features exist as a durable dataset in your datalake
# • You can query them with SQL in Athena
# • You can load them into pandas for modeling
# • You can ingest them into the Feature Store (next cell)

# -----
# 1. Define the S3 path for the engineered feature table
# -----
# We store engineered features under:
# s3://<bucket>/sdwa/parquet/sdwa_pws_quarter_features/
#
# This keeps raw data and engineered data organized in the same structure.

features_table_name = "sdwa_pws_quarter_features"
features_path = f"s3://{bucket}/{parquet_prefix}/{features_table_name}/"

print("Saving engineered features to S3...")
print(f"Target path: {features_path}")

# -----
# 2. Write engineered features to S3 as Parquet
# -----
# WHY PARQUET?
# • Columnar format → fast reads for analytics
# • Compressed → cheaper storage
# • Schema-aware → integrates cleanly with Athena
#
# WHY dataset=True?
# • Allows partitioning and scalable storage
#
# WHY mode="overwrite"?
# • Ensures reproducibility – rerunning the notebook replaces old data

wr.s3.to_parquet(
    df=feat_df,
    path=features_path,
    dataset=True,
    mode="overwrite",
)

print("Engineered features written to S3.")

```

```

# -----
# 3. Register engineered features as an Athena table
# -----
# WHY REGISTER IN ATHENA?
#   • Enables SQL exploration of engineered features
#   • Required for Week-4 modeling workflows
#   • Ensures schema consistency across reruns
#
# We reuse the helper function from Cell 3:
#   safe_create_parquet_table(database, table, path)

print("Registering engineered features as Athena table...")

safe_create_parquet_table(
    database=database,
    table=features_table_name,
    path=features_path,
)

print("Engineered features registered in Athena.")
print("Feature persistence complete.")

```

7. Train/Validation/Test/Production Dataset Splits

This section creates deterministic dataset splits from the engineered feature table:

- 40% training
- 10% validation
- 10% test
- 40% production holdout

Each split is written to S3 and registered in Athena for downstream use.

```

In [ ]: # =====
# 7. Train/Validation/Test/Production Dataset Splits
# =====
# PURPOSE OF THIS CELL:
# -----
# Week 3 requires us to split the engineered feature table into:
#
#   • 40% training data
#   • 10% validation data
#   • 10% test data
#   • 40% production (holdout) data
#
# WHY WE DO THIS:
#   • Training data is used to fit the model
#   • Validation data is used for hyperparameter tuning

```

```

# • Test data is used for final model evaluation
# • Production data simulates "future" unseen data
#
# WHY WE SPLIT BEFORE WEEK 4:
# • Ensures reproducibility
# • Ensures no data leakage
# • Ensures the model is evaluated fairly
#
# WHY WE SAVE SPLITS TO S3:
# • SageMaker training jobs read from S3
# • Athena queries can inspect each split
# • Splits become durable artifacts for the project

# -----
# 1. Shuffle the engineered feature table
# -----
# WHY SHUFFLE?
# • Ensures random distribution across splits
# • Prevents ordering bias (e.g., all early quarters in train)
#
# WHY random_state=42?
# • Guarantees deterministic splits across reruns
# • Ensures reproducibility for grading and debugging

print("Shuffling engineered feature table for deterministic splits...")
df = feat_df.sample(frac=1.0, random_state=42).reset_index(drop=True)

n = len(df)
print(f"Total engineered rows: {n:,}")

# -----
# 2. Compute split boundaries
# -----
# We compute index boundaries based on required percentages:
# • Train: 40%
# • Val: 10%
# • Test: 10%
# • Prod: 40%

train_end = int(0.40 * n)
val_end = train_end + int(0.10 * n)
test_end = val_end + int(0.10 * n)

print(f"Train rows: {train_end:,}")
print(f"Val rows: {val_end - train_end:,}")
print(f"Test rows: {test_end - val_end:,}")
print(f"Prod rows: {n - test_end:,}")

# -----
# 3. Slice the DataFrame into splits
# -----
# These slices are deterministic because:
# • We shuffled with a fixed seed

```

```

# • We use fixed percentage boundaries

train_df = df.iloc[:train_end]
val_df    = df.iloc[train_end:val_end]
test_df   = df.iloc[val_end:test_end]
prod_df   = df.iloc[test_end:]

print("Dataset splits created.")

# -----
# 4. Write each split to S3 as Parquet
# -----
# WHY WRITE SPLITS TO S3?
# • SageMaker training jobs require S3 input
# • Athena queries can inspect each split
# • Splits become durable project artifacts
#
# WHY dataset=True?
# • Allows scalable storage
#
# WHY mode="overwrite"?
# • Ensures reproducibility – rerunning notebook replaces old splits

splits = {
    "train": train_df,
    "val":   val_df,
    "test":  test_df,
    "prod":  prod_df,
}

print("\nWriting dataset splits to S3...")

for name, split in splits.items():

    # Build S3 path for this split
    path = f"s3://{bucket}/{parquet_prefix}{features_table_name}_{name}/"

    print(f" - Writing {name.upper()} split to: {path}")

    wr.s3.to_parquet(
        df=split,
        path=path,
        dataset=True,
        mode="overwrite",
    )

print("All splits written to S3.")

# -----
# 5. Register each split as an Athena table
# -----
# WHY REGISTER SPLITS IN ATHENA?
# • Allows SQL exploration of each split
# • Useful for debugging and validation

```

```
# • Ensures downstream pipelines can query splits directly

print("\nRegistering dataset splits in Athena...")

for name in splits.keys():

    table_name = f"{features_table_name}_{name}"
    path = f"s3://{bucket}/{parquet_prefix}/{table_name}/"

    print(f" - Registering Athena table: {table_name}")

    safe_create_parquet_table(
        database=database,
        table=table_name,
        path=path,
    )

print("All dataset splits registered in Athena.")
print("Dataset splitting complete.")
```

8. Feature Store Ingestion (Offline Store Only)

This section creates a SageMaker Feature Group and ingests the engineered features into the offline Feature Store.

A custom polling loop replaces the unsupported `wait_for_create()` method to ensure compatibility with AWS Academy environments. The Feature Store provides a versioned, queryable repository for Week-4 training.

```
In [ ]: # =====
# 8. Feature Store Ingestion (Offline Store Only)
# =====
# PURPOSE OF THIS CELL:
# -----
# Week 3 requires us to store engineered features in a SageMaker Feature Store.
#
# The Feature Store provides:
# • A central, versioned repository for ML features
# • A consistent source of truth for training and inference
# • A reproducible pipeline for Week-4 model training
#
# We use the OFFLINE store only because:
# • AWS Academy environments often block online store creation
# • Offline store is cheaper and more stable
# • Week-4 training jobs only need offline features
#
# This cell:
# 1. Prepares the engineered DataFrame for Feature Store
# 2. Defines feature metadata (types, names)
# 3. Creates the Feature Group
# 4. Waits for creation (manual polling – required in AWS Academy)
```



```

# 5. Ingests all engineered features into the offline store

import sagemaker
from sagemaker.feature_store.feature_group import FeatureGroup
from sagemaker.feature_store.feature_definition import FeatureDefinition, FeatureTy

# Create a SageMaker session using the same boto3 session as earlier
sm_session = sagemaker.Session(boto_session=session)

# Retrieve the execution role for this notebook
role_arn = sagemaker.get_execution_role()

# -----
# 1. Prepare engineered features for Feature Store
# -----
# Feature Store requires:
#   • A RECORD IDENTIFIER (unique primary key)
#   • An EVENT TIME column (ISO timestamp)
#
# We create:
#   record_id = pwsid + "_" + submissionyearquarter
#   event_time = current UTC timestamp
#
# These fields do NOT change the underlying data – they are metadata required
# by the Feature Store API.

feature_group_name = "sdwa-pws-quarter-features"
record_identifier_name = "record_id"
event_time_feature_name = "event_time"

print("Preparing engineered features for Feature Store ingestion...")

feat_df_fs = feat_df.copy()

# Create a unique primary key for each row
feat_df_fs[record_identifier_name] = (
    feat_df_fs["pwsid"] + "_" + feat_df_fs["submissionyearquarter"]
)

# Create an event timestamp for each row
feat_df_fs[event_time_feature_name] = pd.Timestamp.utcnow().isoformat()

print("Added record_id and event_time columns.")

# -----
# 2. Build feature definitions (schema) for Feature Store
# -----
# Feature Store requires explicit type definitions for each column.
#
# We map:
#   • Numeric columns → FRACTIONAL
#   • Everything else → STRING
#

```

```

# This matches the schema used in Athena and S3.

print("Building feature definitions...")

feature_definitions = []

for col, dtype in feat_df_fs.dtypes.items():

    # Event time must be STRING
    if col == event_time_feature_name:
        ftype = FeatureTypeEnum.STRING

    # Numeric columns become FRACTIONAL
    elif pd.api.types.is_numeric_dtype(dtype):
        ftype = FeatureTypeEnum.FRACTIONAL

    # Everything else becomes STRING
    else:
        ftype = FeatureTypeEnum.STRING

    feature_definitions.append(
        FeatureDefinition(feature_name=col, feature_type=ftype)
    )

print(f"Feature definitions created for {len(feature_definitions)} columns.")

# -----
# 3. Create the Feature Group
# -----
# This registers the schema and metadata with SageMaker Feature Store.
#
# enable_online_store=False ensures:
#   • No DynamoDB table is created
#   • No real-time lookup endpoint is created
#   • Offline-only storage in S3 is used (recommended for AWS Academy)

print(f"Creating Feature Group '{feature_group_name}'...")

feature_group = FeatureGroup(
    name=feature_group_name,
    sagemaker_session=sm_session,
    feature_definitions=feature_definitions,
)

feature_group.create(
    s3_uri=features_path,                                # Store offline features in same S3 f
    record_identifier_name=record_identifier_name,
    event_time_feature_name=event_time_feature_name,
    role_arn=role_arn,
    enable_online_store=False,                            # Offline-only mode
)

print("Feature Group creation initiated.")

```

```

# -----
# 4. Poll for Feature Group creation (AWS Academy fix)
# -----
# WHY WE DO THIS:
#   • The SDK version in AWS Academy does NOT support wait_for_create()
#   • We must manually poll the Feature Group status
#   • Ingestion will fail if we try to ingest before creation completes

def wait_for_fg_creation(fg):
    """
    Polls the Feature Group status until it becomes 'Created'.
    Raises an error if creation fails.
    """
    status = fg.describe().get("FeatureGroupStatus")

    while status == "Creating":
        print("Waiting for Feature Group creation...")
        time.sleep(5)
        status = fg.describe().get("FeatureGroupStatus")

    if status != "Created":
        raise RuntimeError(f"Feature Group creation failed: {status}")

    print("Feature Group created successfully.")

wait_for_fg_creation(feature_group)

# -----
# 5. Ingest engineered features into the offline store
# -----
# WHY max_workers=4?
#   • Parallel ingestion improves performance
#
# WHY wait=True?
#   • Ensures ingestion completes before moving to Week-4 tasks

print("Ingesting engineered features into Feature Store (offline store)...")

feature_group.ingest(
    data_frame=feat_df_fs,
    max_workers=4,
    wait=True,
)

print("Feature Store ingestion complete.")

```

In []: